

Department of Computer Science
CS403NOD – Node.js

Lab Assignment – 05

Simulating a Food Delivery Tracker — Callbacks, Promises & Async/Await

Semester: BCA VII

Lab No.: 05

Objective

After completing this lab, you should be able to:

- Implement the same async logic three different ways: callbacks, Promises, and async/await.
- Understand Promise states (pending, fulfilled, rejected) through a real tracking scenario.
- Chain multiple sequential async steps correctly, with proper error handling.
- Handle errors using both `.catch()` and `try/catch`.
- Run multiple async operations concurrently using `Promise.all()`.
- Connect the behavior of this code back to the Event Loop (Theory Class).

Tasks

Task 1 – Project Setup

Create the following folder structure.

```
NodeJS-Lab/  
├── Lab-05/  
│   ├── callback-version.js  
│   ├── promise-version.js  
│   ├── chaining-version.js  
│   ├── async-await-version.js  
│   ├── concurrent-orders.js  
│   ├── reflection-notes.txt  
│   └── README.md
```

Task 2 – Callback Version (Feel The Pain)

Create

callback-version.js

Start with a single order using a callback.

```
function placeOrderCallback(item, callback) {  
  console.log(`Order placed: ${item}`);  
  setTimeout(() => {  
    callback(`${item} is out for delivery!`);  
  }, 2000);  
}  
  
placeOrderCallback('Pizza', (message) => {  
  console.log(message);  
});
```

Now extend it: write `trackOrderCallback()` and `confirmDeliveryCallback()` in the same style (each using `setTimeout` and a callback), and nest all three so a single order goes through Place → Track → Confirm, one inside the other.

Run it, and look carefully at how far your code has drifted to the right by the third level. Take a screenshot of both the code and its output.

Save it as

callback-output.png

Task 3 – Promise Version (Fix The Pain)

Create

promise-version.js

Rewrite `placeOrder` as a function that returns a Promise instead of taking a callback.

```
function placeOrder(item) {
  return new Promise((resolve, reject) => {
    console.log(`Order placed: ${item}`);
    setTimeout(() => {
      const success = Math.random() > 0.2; // ~80% chance of success
      if (success) {
        resolve(`${item} is out for delivery!`);
      } else {
        reject(`Sorry, the restaurant could not prepare ${item}`);
      }
    }, 2000);
  });
}

placeOrder('Burger')
  .then((message) => console.log(message))
  .catch((error) => console.log(error));
```

Run the file at least 5 times in a row. Because of the random chance, you should see BOTH the fulfilled (`.then`) and rejected (`.catch`) paths happen across your runs. Take a screenshot showing at least one of each.

Task 4 – Promise Chaining (Order Lifecycle)

Create

chaining-version.js

Write three Promise-returning functions — `placeOrder(item)`, `trackOrder(item)`, and `confirmDelivery(item)` — each simulating one stage with a 1-second delay, printing its own stage message (“Order Placed”, “Preparing”, “Out for Delivery”).

Chain all three together so the item flows through every stage in order, ending with a final “Delivered” message, with exactly ONE `.catch()` at the end handling any failure from any stage.

```
placeOrder('Pasta').then(...).then(...).then(...).catch(...)
```

Run it and confirm all four stage messages print in the correct order, with a real delay between each.

Task 5 – Async/Await Refactor

Create

async-await-version.js

Reuse the same `placeOrder()`, `trackOrder()`, and `confirmDelivery()` functions from Task 4 (copy or require them), but this time consume them using an `async` function with `await`, wrapped in `try/catch` instead of `.then()/catch()`.

Confirm the output is identical in meaning to Task 4 — same four stages, same final result — just written differently.

Hint: your `async` function should read almost like a plain to-do list — `await placeOrder`, `await trackOrder`, `await confirmDelivery`, each on its own line.

Task 6 – Concurrent Orders with Promise.all()

Create

concurrent-orders.js

Simulate placing three different orders (e.g. Pizza, Burger, Coffee) at the same time, each with a slightly different random delay, using Promise.all().

```
async function orderMultiple() {  
  console.log('Placing 3 orders at once...');  
  const results = await Promise.all([  
    placeOrder('Pizza'),  
    placeOrder('Burger'),  
    placeOrder('Coffee')  
  ]);  
  console.log(results);  
}
```

Time how long the whole thing takes (console.time() / console.timeEnd() if you want to be precise, or just observe roughly).

Question to answer in your lab record: does Promise.all() take the SUM of all three delays, or roughly the LONGEST single delay? Test it and explain why, in your own words.

Task 7 – Connect It Back To The Event Loop

Create a text file named

reflection-notes.txt

Answer the following in your own words (4–5 sentences each):

- While your order's setTimeout is “waiting,” is the Node.js process frozen? Use what you learned in Theory Class to explain what's actually happening.
- Why does Promise.all() finish faster than running each order one after another with three separate await calls?

README.md

Update your README.md to also include:

- A one-line summary of what each of the 5 .js files in this lab demonstrates
- Lab Number: 05
- Date

GitHub Submission

Use your existing repository — no need to create a new one.

Step 1

Open Terminal inside NodeJS-Lab.

Step 2

Run the following commands.

git add .

git commit -m "Lab 05 Submission - Async JavaScript (Food Delivery Tracker)"

git push

Google Classroom Submission

Submit

- GitHub Repository Link
- (Optional) ZIP File

If You Face Any Problem

If you are unable to complete any task, **do not leave it incomplete without explanation.**

Create a section in your **README.md** named:

Problems Faced

Mention:

- Task Number
- Error / Issue
- What you tried to solve it
- Screenshot (if applicable)

Example

Problems Faced

Task No.: Task 4

Issue:

My chain printed 'undefined' at the Confirm Delivery stage instead of the item name.

Attempted Solution:

Realized my trackOrder() function wasn't returning anything inside its setTimeout/resolve. Added the missing return value and the chain worked correctly.

Students who clearly document genuine problems will receive guidance during the next lab session.